LinkedHashSet<E> [It is an ordered version of HashSet]

public class LinkedHashSet<E> extends HashSet<E> implements SequencedSet, Clonable, Serializable

It is a predefined class in java.util package under Set interface and introduced from java 1.4v.

It is the sub class of HashSet class.

It is an orderd version of HashSet that maintains a doubly linked list across all the elements that is the reason It implements from
SequenceSet from Java 21V

It internally uses Hashtable and doubly LinkedList data structures.

We should use LinkedHashSet class when we want to maintain an order.

*When we iterate the elements through HashSet the order will be un-predictable, while when we iterate the elements through LinkedHashSet then the order will be same as they were inserted in the collection.

It accepts hetrogeneous and null value is allowed.

It has same constructor as HashSet class.

//Programs :

```java
package com.ravi.linkedhashset;

import java.util.LinkedHashSet;

public class LinkedHashSetDemo1
{
    public static void main(String[] args)
    {
         LinkedHashSet<String> lhs = new LinkedHashSet<>();
         lhs.add("Ravi");
         lhs.add("Vijay");
         lhs.add("Ravi");
         lhs.add("Ajay");
         lhs.add("Pawan");
         lhs.add("Shiva");
         lhs.add(null);
         lhs.add("Ganesh");
         lhs.forEach(str -> System.out.println(str));

    }

}
```

```java
 package com.ravi.linkedhashset;

 import java.util.LinkedHashSet;

 public class LinkedHashSetDemo2 {

    public static void main(String[] args)
    {
        LinkedHashSet<Integer> linkedHashSet = new LinkedHashSet<>();

         linkedHashSet.add(10);
         linkedHashSet.add(5);
         linkedHashSet.add(15);
         linkedHashSet.add(20);
         linkedHashSet.add(5);

         System.out.println("LinkedHashSet elements: " + linkedHashSet);

         System.out.println("LinkedHashSet size: " + linkedHashSet.size());

         int elementToCheck = 15;
         if (linkedHashSet.contains(elementToCheck))
        {
             IO.println(elementToCheck + " is present in the LinkedHashSet.");
         }
        else
        {
             IO.println(elementToCheck + " is not present in the LinkedHashSet.");
         }

         int elementToRemove = 10;
         linkedHashSet.remove(elementToRemove);
         IO.println("After removing " + elementToRemove + ", LinkedHashSet elements: " +
 linkedHashSet);

               linkedHashSet.clear();
         System.out.println("After clearing, LinkedHashSet elements: " + linkedHashSet);
 //[]

    }

 }
```

LinkedHashMap<K,V> [It is an ordered version of HashMap<K,V>]

public class LinkedHashMap<K,V> extends HashMap<K,V> implements SequenceMap<K,V>

It is a predefined class available in java.util package under Map interface available from 1.4.

It is the sub class of HashMap class.

It maintains insertion order so, from Java 21v it implements from SequencedMap. It contains a doubly linked with the elements or nodes so It will iterate more slowly in comparison to HashMap.

It uses Hashtable and doubly LinkedList data structure.

If We want to fetch the elements in the same order as they were inserted in the Map then we should go with LinkedHashMap.

It accepts one null key and multiple null values.

It is not synchronized.

It has also 4 constructors same as HashMap

1) LinkedHashMap hm1 = new LinkedHashMap();
   will create a LinkedHashMap with default capacity 16 and load factor 0.75

2) LinkedHashMap hm1 = new LinkedHashMap(int initialCapacity);

3) LinkedHashMap hm1 = new LinkedHashMap(int initialCapacity, float loadFactor);

4) LinkedHashMap hm1 = new LinkedHashMap(Map m);

//Prorams :

```java
package com.ravi.linked_hash_map;

import java.util.LinkedHashMap;

public class LinkedHashMapDemo1 {

    public static void main(String[] args)
    {
        LinkedHashMap<Integer, String> map = new LinkedHashMap<>();
        map.put(1, "Raj");
        map.putIfAbsent(1, "Ravi");
        map.put(5, "Scott");
        map.put(3, null);
        map.put(4, null);
        map.put(2, "Smith");
        map.put(null, "John");
        map.put(null, "Alen");
        map.forEach((k,v)-> IO.println(k+" :"+v));

    }

}
```

SortedSet<E>

It is the sub interface of Set<E> interface available from JDK 1.2V

As we know we can't perform sorting operation on HashSet and LinkedHashSet object by using Collections.sort(List<E> list)

In order to provide automatic sorting facility with Set interface, java software people has provided SortedSet<E> interface.

By default It will provide defult natural sorting order.

It is a Set that maintains its elements in a sorted (ascending) order, either by their natural ordering (**Comparable**) or using a specified **Comparator.**

TreeSet<E>

public class TreeSet<E> extends AbstractSet<E> implements NavigableSet<E> , Cloneable, Serializable

* It is an implemented class of Set interface available from JDK 1.2V
* It does not accept duplicate objects.
* It uses Red Black Tree Data structure.
* It does not accept null otherwise we will get NullPonterException
* It does not accept **non comparable objects [ClassCastException]**
* By default, It provides default natural sorting order using Comparable.
* We can also sort by using Comparator through TreeSet constructor.
* Whenever we call add() method of TreeSet, internally it is calling compareTo() method of Comparable<T>

It contains 4 types of constructors :

1) TreeSet t1 = new TreeSet();
   create an empty TreeSet object, elements will be inserted in using Comparable using compareTo() method internally.

2) TreeSet t2 = new TreeSet(Comparator<T> c);
   create an empty TreeSet object, elements will be inserted in using Comparator using compare() internally.

3) TreeSet t3 = new TreeSet(Collection c);
   loose coupling.

4) TreeSet t4 = new TreeSet(SortedSet s);
   We can merge two TreeSet object to copy the data.

//Programs :

```java
 package com.ravi.tree_set_demo;

 import java.util.TreeSet;

 public class TreeSetDemo1
 {
     public static void main(String[] args)
     {
         TreeSet<Object> ts1 = new TreeSet<>();
          ts1.add(9);
          ts1.add(1); //compareTo()
          ts1.add(5);
          ts1.add(4);
          ts1.add(2);
          //ts1.add(null); //NPE
          //ts1.add(true); //ClassCastException
          IO.print(ts1);
     }
 }
```

```java
 package com.ravi.tree_set_demo;

 import java.util.Iterator;
 import java.util.TreeSet;

 public class TreeSetDemo2
 {
     public static void main(String[] args)
     {
         TreeSet<String> ts2 = new TreeSet<>();
         ts2.add("Mango");
         ts2.add("Grapes");
         ts2.add("Apple");
         ts2.add("Orange");



         IO.println("In ascending Order :");
         ts2.forEach(IO::println);

         IO.println("In descending Order :");

         Iterator<String> descendingIterator = ts2.descendingIterator();
         descendingIterator.forEachRemaining(IO::println);

         IO.print(ts2);


     }

 }

 /*
  *
 Note :- descendingIterator() is a predefined method of TreeSet class which will
 traverse in the descending order and return type of this method is
 Iterator interface, method is available from JDK 1.6

 public Iterator descendingIterator()
```

```java
package com.ravi.tree_set_demo;

import java.util.TreeSet;

record Employee(Integer id, String name) implements Comparable<Employee>
{
    @Override
    public int compareTo(Employee other)
    {
        return this.name().compareTo(other.name());
    }
}
public class TreeSetDemo3
{
    public static void main(String[] args)
    {
        TreeSet<Employee> ts1 = new TreeSet<>();
        ts1.add(new Employee(333, "Scott"));
        ts1.add(new Employee(111, "Zuber")); //compareTo()
        ts1.add(new Employee(222, "Aryan"));

        for(Employee employee : ts1)
        {
            IO.println(employee);
        }

    }

}
```

```java
package com.ravi.tree_set_demo;

import java.util.TreeSet;

record Student(Integer id, String name)
{
}

public class TreeSetDemo4
{
    public static void main(String[] args)
    {
       TreeSet<Student> students = new TreeSet<>((s1, s2)-> Integer.compare(s1.id(), s2.id()));
       students.add(new Student(222, "Raj"));
       students.add(new Student(111, "Zuber"));
       students.add(new Student(333, "Aryan"));

       students.forEach(std -> IO.println(std));

    }

}
```

```java
 package com.ravi.tree_set_demo;

 import java.util.TreeSet;

 public class TreeSetDemo5
 {
     public static void main(String[] args)
     {
         TreeSet<StringBuffer> ts1 = new TreeSet<>((sb1, sb2)-> sb2.compareTo(sb1));
         ts1.add(new StringBuffer("B"));
         ts1.add(new StringBuffer("A"));
         ts1.add(new StringBuffer("C"));

         IO.println(ts1);

         TreeSet<StringBuilder> ts2 = new TreeSet<>();
         ts2.add(new StringBuilder("B"));
         ts2.add(new StringBuilder("A"));
         ts2.add(new StringBuilder("C"));

         IO.println(ts2);

     }
 }
```

Note : Starting from Java 11, both StringBuilder and StringBuffer do implement the Comparable interface

---

```java
package com.ravi.tree_set_demo;

import java.util.ArrayList;
import java.util.TreeSet;

public class TreeSetDemo6 {

     public static void main(String[] args)
     {
          TreeSet<Character> ts1 = new TreeSet<>();
          ts1.add('C');
          ts1.add('B');
          ts1.add('A');
        IO.println(ts1);

        TreeSet<Character> ts2 = new TreeSet<>(ts1);
        IO.println(ts2);

        ArrayList<String> listOfCity = new ArrayList<>();
        listOfCity.add("Hyderabad");
        listOfCity.add("Hyderabad");
        listOfCity.add("Pune");
        listOfCity.add("Mumbai");
        listOfCity.add("Ajmer");
        listOfCity.add("Surat");

        TreeSet<String> cities = new TreeSet<>(listOfCity);
        IO.println(cities);

     }

}
```

```java
package com.ravi.tree_set_demo;

import java.util.TreeSet;

record Product(Integer id, String name, Double price)
{
}

public class TreeSetDemo7
{
    public static void main(String[] args)
    {
        TreeSet<Product> ts1 = new TreeSet<>((p1, p2)-> Integer.compare(p1.id(), p2.id()));
        ts1.add(new Product(333, "Mobile", 56789D));
        ts1.add(new Product(111, "Laptop", 46789D));
        ts1.add(new Product(222, "Camera", 96789D));

        IO.println("Sorting based on the Id :");
        ts1.forEach(IO::println);

        TreeSet<Product> ts2 = new TreeSet<>((p1, p2)-> Double.compare(p1.price(), p2.price()));
        ts2.add(new Product(333, "Mobile", 56789D));
        ts2.add(new Product(111, "Laptop", 46789D));
        ts2.add(new Product(222, "Camera", 96789D));

        IO.println("Sorting based on the Price :");
        ts2.forEach(IO::println);

    }

}
```

SortedMap<K,V>

It is the sub interface of Map<K,V> available from JDK 1.2V.

It maintains its keys in a sorted order.

By default It provides according to the natural ordering of the keys or by a custom Comparator provided at map creation time.

TreeMap<K,V>

public class TreeMap<K,V> extends AbstractMap<K,V> implements NavigableMap<K,V> , Clonable, Serializable

It is a predefined class avaialble in java.util package under Map interface available for 1.2V.

It is a sorted map that means it will sort the elements by natural sorting order based on the key (Comparable) or by using Comparator interface as a constructor parameter.

It does not allow non comparable keys.(ClassCastException)

It does not accept null key (NPE) but null values are allowed.

It uses Red black tree data structure.

TreeMap implements NavigableMap and NavigableMap extends SortedMap. SortedMap extends Map interface.

TreeMap contains 4 types of Constructors :

1) TreeMap tm1 = new TreeMap(); //creates an empty TreeMap

2) TreeMap tm2 = new TreeMap(Comparator cmp); //user defined sorting logic

3) TreeMap tm3 = new TreeMap(Map m); //loose Coupling

4) TreeMap tm4 = new TreeMap(SortedMap m);

//Programs

```java
package com.ravi.tree_map_demo;

import java.util.TreeMap;

public class TreeMapDemo1 {

    public static void main(String[] args)
    {
        TreeMap<Object,String> t = new TreeMap<>();
        t.put(4,"Ravi");
        t.put(7,"Aswin");
        t.put(2,"Ananya");
        t.put(1,"Dinesh");
        t.put(9,"Ravi");
        t.put(3,"Ankita");
        t.put(5,null);
        //t.put("six", "Xyz");
        //t.put(null, "abc");
        IO.println(t);
    }

}
```